In [1]:
```
# Online Retail Sales & Customer Analytics:--

# 1. Project Introduction:-

#The Online Retail Sales & Customer Analytics project aims to analyze transactio
#customer purchasing behavior, product performance, country-wise revenue, cancel
#The project uses Python, Pandas, NumPy, Matplotlib, Seaborn, SQL, and Power BI
#insights and interactive dashboards.

# 2. Business Objective:-

#- Analyze overall sales and revenue performance.
#- Identify top-performing products and countries.
#- Understand customer purchasing patterns and repeat purchases.
#- Analyze monthly revenue trends.
#- Examine cancelled transactions and data quality issues.
#- Develop actionable business recommendations.
```

In [2]:
```
# Import Required Libraries:-

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

In [3]:
```
# Load Dataset:-

df = pd.read_csv(r"C:\Users\Dell\OneDrive\Documents\Online Retail Sales & Custom
print("Dataset loaded successfully!")
print("Dataset Shape:-",df.shape)
```

```
Dataset loaded successfully!
Dataset Shape:- (1067371, 8)
```

In [4]:
```
# Display the first 5 rows:-
df.head()
```

Out[4]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.0	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.0	United Kingdom

In [5]:
```
# Check the number of rows and columns:-
print("Rows:-",df.shape[0])
print("Column:-",df.shape[1])
```

```
Rows:- 1067371
Column:- 8
```

In [6]:
```
# Check all column names:-
print(df.columns.tolist())
```

```
['Invoice', 'StockCode', 'Description', 'Quantity', 'InvoiceDate', 'Price', 'Cust
omer ID', 'Country']
```

In [7]:
```
# Check data type:-
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1067371 entries, 0 to 1067370
Data columns (total 8 columns):
 #   Column       Non-Null Count    Dtype  
---  ------       --------------    -----  
 0   Invoice      1067371 non-null  object 
 1   StockCode    1067371 non-null  object 
 2   Description  1062989 non-null  object 
 3   Quantity     1067371 non-null  int64  
 4   InvoiceDate  1067371 non-null  object 
 5   Price        1067371 non-null  float64
 6   Customer ID  824364 non-null   float64
 7   Country      1067371 non-null  object 
dtypes: float64(2), int64(1), object(5)
memory usage: 65.1+ MB
```

In [8]:
```
# Check  missing values:-
print(df.isnull().sum())
```

```
Invoice             0
StockCode           0
Description      4382
Quantity            0
InvoiceDate         0
Price               0
Customer ID    243007
Country             0
dtype: int64
```

In [9]:
```
# Check duplicate records:-
print("Duplicate rows:-",df.duplicated().sum())
```

```
Duplicate rows:- 34335
```

In [10]:
```
# Check basic statistics:-
df.describe(include="all").T
```

Out[10]:

	count	unique	top	freq	mean	std	min
Invoice	1067371	53628	537434	1350	NaN	NaN	NaN
StockCode	1067371	5305	85123A	5829	NaN	NaN	NaN
Description	1062989	5698	WHITE HANGING HEART T-LIGHT HOLDER	5918	NaN	NaN	NaN
Quantity	1067371.0	NaN	NaN	NaN	9.938898	172.705794	-80995.0
InvoiceDate	1067371	47635	2010-12-06 16:57:00	1350	NaN	NaN	NaN
Price	1067371.0	NaN	NaN	NaN	4.649388	123.553059	-53594.36
Customer ID	824364.0	NaN	NaN	NaN	15324.638504	1697.46445	12346.0
Country	1067371	43	United Kingdom	981330	NaN	NaN	NaN

In [11]:
```
# Data Cleaning:-
# Handling Duplicate Records
```

In [12]:
```
print("Duplicate rows before cleaning:-",df.duplicated().sum())
```

```
Duplicate rows before cleaning:- 34335
```

In [13]:
```
df = df.drop_duplicates()
print("Shape after removing duplicates:-",df.shape)
print("Duplicate rows remaining:-",df.duplicated().sum())
```

```
Shape after removing duplicates:- (1033036, 8)
Duplicate rows remaining:- 0
```

In [14]:
```
# Check the updated missing values:-
print(df.isnull().sum())
```

```
Invoice             0
StockCode           0
Description      4275
Quantity            0
InvoiceDate         0
Price               0
Customer ID    235151
Country             0
dtype: int64
```

In [15]:
```python
# Handling Missing Discription Values:-
print("Missing Description:-",df["Description"].isnull().sum())
print("Percentage Missing:-",round(df["Description"].isnull().mean() * 100, 2),"
```

```
Missing Description:- 4275
Percentage Missing:- 0.41 %
```

In [16]:
```python
# Remove rows where Description is missing:-
df = df.dropna(subset=["Description"])

print("Shape after cleaning:-",df.shape)
print("Missing Description:-",df["Description"].isnull().sum())
```

```
Shape after cleaning:- (1028761, 8)
Missing Description:- 0
```

In [17]:
```python
# Check Customer ID missing values:-
print("Missing Customer ID:-", df["Customer ID"].isnull().sum())
print("Missing Customer ID Percentage:-",round(df["Customer ID"].isnull().mean()
```

```
Missing Customer ID:- 230876
Missing Customer ID Percentage:- 22.44 %
```

In [18]:
```python
# Checking Invalid Quantity and Price Values:-
df[["Quantity", "Price"]].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1028761 entries, 0 to 1067370
Data columns (total 2 columns):
 #   Column    Non-Null Count    Dtype
---  ------    --------------    -----
 0   Quantity  1028761 non-null  int64
 1   Price     1028761 non-null  float64
dtypes: float64(1), int64(1)
memory usage: 23.5 MB
```

In [19]:
```python
# Check negative and zero values:-
print("Zero Quantity:-",(df["Quantity"]== 0).sum())
print("Negative Quantity:-",(df["Quantity"] < 0).sum())

print("Zero Price:-",(df["Price"]== 0).sum())
print("Negative Price:-",(df["Price"] < 0).sum())
```

```
Zero Quantity:- 0
Negative Quantity:- 19863
Zero Price:- 1739
Negative Price:- 5
```

In [20]:
```python
# Check cancelled invoices:-
print("Cancelled invoices:-",df["Invoice"].astype(str).str.startswith("C").sum()
```

```
Cancelled invoices:- 19104
```

In [21]:
```
# View sample records with negative quantities:-
df[df["Quantity"] < 0].head(10)
```

Out[21]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Count
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16321.0	Austra
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16321.0	Austra
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16321.0	Austra
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16321.0	Austra
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16321.0	Austra
183	C489449	21871	SAVE THE PLANET MUG	-12	2009-12-01 10:33:00	1.25	16321.0	Austra
184	C489449	84946	ANTIQUE SILVER TEA GLASS ETCHED	-12	2009-12-01 10:33:00	1.25	16321.0	Austra
185	C489449	84970S	HANGING HEART ZINC T-LIGHT HOLDER	-24	2009-12-01 10:33:00	0.85	16321.0	Austra
186	C489449	22090	PAPER BUNTING RETRO SPOTS	-12	2009-12-01 10:33:00	2.95	16321.0	Austra
196	C489459	90200A	PURPLE SWEETHEART BRACELET	-3	2009-12-01 10:44:00	4.25	17592.0	Unit Kingdo

In [22]:
```
# Investigating Invalid Price Values
df[df["Price"] < 0]
```

Out[22]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID
179403	A506401	B	Adjust bad debt	1	2010-04-29 13:36:00	-53594.36	NaN
276274	A516228	B	Adjust bad debt	1	2010-07-19 11:24:00	-44031.79	NaN
403472	A528059	B	Adjust bad debt	1	2010-10-20 12:04:00	-38925.87	NaN
825444	A563186	B	Adjust bad debt	1	2011-08-12 14:51:00	-11062.06	NaN
825445	A563187	B	Adjust bad debt	1	2011-08-12 14:52:00	-11062.06	NaN

In [23]:

```
# Check the Zero-price records:-
df[df["Price"] == 0].head(20)
```

Out[23]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Coun
263	489464	21733	85123a mixed	-96	2009-12-01 10:52:00	0.0	NaN	Unit Kingdo
283	489463	71477	short	-240	2009-12-01 10:52:00	0.0	NaN	Unit Kingdo
284	489467	85123A	21733 mixed	-192	2009-12-01 10:53:00	0.0	NaN	Unit Kingdo
3162	489660	35956	lost	-1043	2009-12-01 17:43:00	0.0	NaN	Unit Kingdo
3168	489663	35605A	damages	-117	2009-12-01 18:02:00	0.0	NaN	Unit Kingdo
4538	489820	21133	invcd as 84879?	-720	2009-12-02 13:23:00	0.0	NaN	Unit Kingdo
4674	489825	22076	6 RIBBONS EMPIRE	12	2009-12-02 13:34:00	0.0	16126.0	Unit Kingdo
5904	489861	DOT	DOTCOM POSTAGE	1	2009-12-02 14:50:00	0.0	NaN	Unit Kingdo
6556	489899	79323GR	sold as gold	-954	2009-12-03 09:41:00	0.0	NaN	Unit Kingdo
6781	489998	48185	DOOR MAT FAIRY CAKE	2	2009-12-03 11:19:00	0.0	15658.0	Unit Kingdo
6911	490007	84347	21494	-720	2009-12-03 12:09:00	0.0	NaN	Unit Kingdo
9308	490130	21493	lost?	-600	2009-12-03 18:28:00	0.0	NaN	Unit Kingdo
16107	490727	M	Manual	1	2009-12-07 16:38:00	0.0	17231.0	Unit Kingdo
17427	490765	21450	damaged	-31	2009-12-08 10:59:00	0.0	NaN	Unit Kingdo
17428	490766	51008	wet	-200	2009-12-08 11:06:00	0.0	NaN	Unit Kingdo
18738	490961	22065	CHRISTMAS PUDDING TRINKET POT	1	2009-12-08 15:25:00	0.0	14108.0	Unit Kingdo
18739	490961	22142	CHRISTMAS CRAFT WHITE FAIRY	12	2009-12-08 15:25:00	0.0	14108.0	Unit Kingdo
20970	491065	84855	MIA	-108	2009-12-09 11:59:00	0.0	NaN	Unit Kingdo
28111	491732	79030A	smashed	-350	2009-12-14 10:00:00	0.0	NaN	Unit Kingdo

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Coun
31993	491971	85042	ANTIQUE LILY FAIRY LIGHTS	1	2009-12-14 18:37:00	0.0	NaN	Unit Kingd

In [24]:
```
print("Zero-price records:-",(df["Price"] == 0).sum())
print("Negative-price records:-",(df["Price"] < 0).sum())
```

```
Zero-price records:- 1739
Negative-price records:- 5
```

In [25]:
```
# Remove only negative-price records:-
df = df[df["Price"] >= 0].copy()

print("Shape after cleaning:-",df.shape)
print("Negative prices remaining:-",(df["Price"] < 0).sum())
```

```
Shape after cleaning:- (1028756, 8)
Negative prices remaining:- 0
```

In [26]:
```
# Identifying Cancelled Transactions:-
df["Invoice"] = df["Invoice"].astype(str)

df["IsCancelled"] = df["Invoice"].str.startswith("C")

print(df["IsCancelled"].value_counts())
```

```
IsCancelled
False    1009652
True       19104
Name: count, dtype: int64
```

In [27]:
```
# Check the cancellation summary:-
print("Cancelled transaction rows:-",df["IsCancelled"].sum())

print("Normal transaction rows:-",(~df["IsCancelled"]).sum())
```

```
Cancelled transaction rows:- 19104
Normal transaction rows:- 1009652
```

In [28]:
```
# To Count unique cancelled invoices:-
cancelled_invoices = df.loc[df["IsCancelled"],"Invoice"].nunique()
print("Unique cancelled invoices:-",cancelled_invoices)
```

```
Unique cancelled invoices:- 8292
```

In [29]:
```
# Calculating Revenue:-
# Revenue = Quantity * Price
```

In [30]:
```
df["Revenue"] = df["Quantity"] * df["Price"]
print(df[["Quantity","Price","Revenue"]].head())
```

```
   Quantity  Price  Revenue
0        12   6.95     83.4
1        12   6.75     81.0
2        12   6.75     81.0
3        48   2.10    100.8
4        24   1.25     30.0
```

In [31]:
```
# Converting InvoiceDate to Datetime:-
print(df["InvoiceDate"].dtype)
```

```
object
```

In [32]:
```
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"],errors="coerce")
print("InvoiceDate datatype:-",df["InvoiceDate"].dtype)
print("Missing InvoiceDate:-",df["InvoiceDate"].isnull().sum())
```

```
InvoiceDate datatype:- datetime64[ns]
Missing InvoiceDate:- 0
```

In [33]:
```
# Extract Year and Month:-
df["Year"] = df["InvoiceDate"].dt.year
df["Month"] = df["InvoiceDate"].dt.month
df["YearMonth"] = df["InvoiceDate"].dt.to_period("M").astype(str)

df[["InvoiceDate", "Year", "Month", "YearMonth"]].head()
```

Out[33]:

	InvoiceDate	Year	Month	YearMonth
0	2009-12-01 07:45:00	2009	12	2009-12
1	2009-12-01 07:45:00	2009	12	2009-12
2	2009-12-01 07:45:00	2009	12	2009-12
3	2009-12-01 07:45:00	2009	12	2009-12
4	2009-12-01 07:45:00	2009	12	2009-12

In [34]:
```
import os

os.makedirs(r"C:\Users\Dell\OneDrive\Documents\Online Retail Sales & Customer An

df.to_csv(r"C:\Users\Dell\OneDrive\Documents\Online Retail Sales & Customer Anal

print("Cleaned dataset saved successfully!")
```

```
Cleaned dataset saved successfully!
```

In [35]:
```
# Exploratory Data Analysis (EDA)
# Overall Business Performance:-
```

In [36]:
```
# Calculate the main KPIs:-

# Total transaction records
total_records = len(df)

# Unique invoices
total_invoices = df["Invoice"].nunique()

# Unique customers (excludng missing IDs)
total_customers = df["Customer ID"].nunique()
```

```
# Unique Products
total_products = df["StockCode"].nunique()

# Total countries
total_countries = df["Country"].nunique()

# Total revenue, including returns and cancellations
total_revenue = df["Revenue"].sum()

print("Total Records:-",total_records)
print("Total Invoices:-",total_invoices)
print("Unique Customers:-",total_customers)
print("Unique Products:-",total_products)
print("Countries:-",total_countries)
print("Net Transaction Revenue:-",round(total_revenue, 2))
```

```
Total Records:- 1028756
Total Invoices:- 49348
Unique Customers:- 5942
Unique Products:- 4950
Countries:- 43
Net Transaction Revenue:- 19014209.84
```

In [37]:
```
# Check the results:-
df[["Invoice", "StockCode", "Quantity", "Price", "Revenue", "Country"]].head(10)
```

Out[37]:

	Invoice	StockCode	Quantity	Price	Revenue	Country
0	489434	85048	12	6.95	83.4	United Kingdom
1	489434	79323P	12	6.75	81.0	United Kingdom
2	489434	79323W	12	6.75	81.0	United Kingdom
3	489434	22041	48	2.10	100.8	United Kingdom
4	489434	21232	24	1.25	30.0	United Kingdom
5	489434	22064	24	1.65	39.6	United Kingdom
6	489434	21871	24	1.25	30.0	United Kingdom
7	489434	21523	10	5.95	59.5	United Kingdom
8	489435	22350	12	2.55	30.6	United Kingdom
9	489435	22349	12	3.75	45.0	United Kingdom

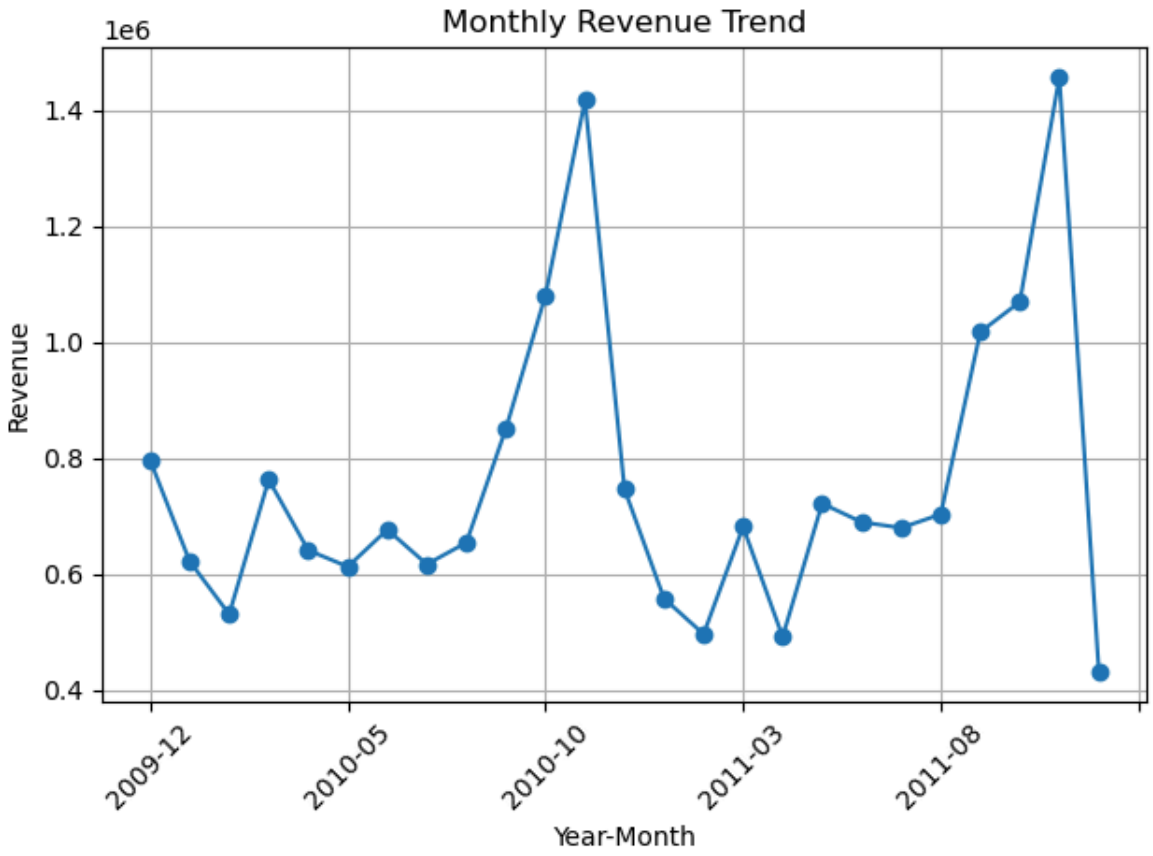
In [38]:
```
# Monthly Revenue Trend Analysis:-
monthly_revenue = df.groupby("YearMonth")["Revenue"].sum()

monthly_revenue
```

Out[38]:
```
YearMonth
2009-12     796648.500
2010-01     622479.502
2010-02     531265.366
2010-03     763247.241
2010-04     641521.052
2010-05     613270.720
2010-06     677073.870
2010-07     617365.480
2010-08     654774.390
2010-09     851105.961
2010-10    1080611.480
2010-11    1416697.202
2010-12     746723.610
2011-01     558448.560
2011-02     497026.410
2011-03     682013.980
2011-04     492367.841
2011-05     722094.100
2011-06     689977.230
2011-07     680156.991
2011-08     703510.580
2011-09    1017596.682
2011-10    1069368.230
2011-11    1456145.800
2011-12     432719.060
Name: Revenue, dtype: float64
```

In [39]:
```
# (1) Visualization:-

monthly_revenue.plot(kind = "line", marker = "o")
plt.title("Monthly Revenue Trend")
plt.xlabel("Year-Month")
plt.ylabel("Revenue")
plt.xticks(rotation = 45)
plt.grid()
plt.tight_layout()
plt.show()
```



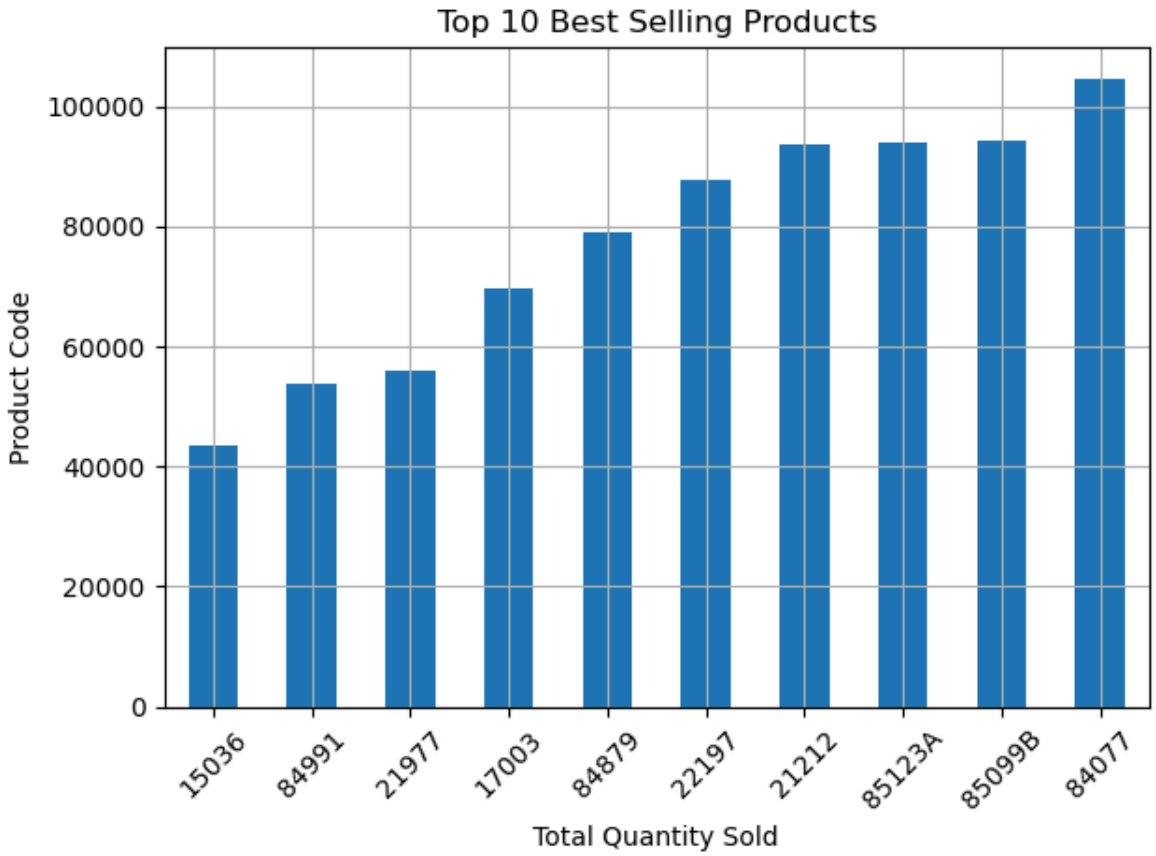
In [40]:
```
# Top 10 Best-Selling Products:-
top_products = df.groupby("StockCode")["Quantity"].sum()

# Get top 10 products
top_products = top_products.sort_values(ascending = False).head(10)

print(top_products)
```

```
StockCode
84077     104546
85099B     94151
85123A     94123
21212      93774
22197      87731
84879      78978
17003      69639
21977      55842
84991      53678
15036      43463
Name: Quantity, dtype: int64
```

In [41]:
```
# (2) Visualization:-
top_products.sort_values().plot(kind = "bar")
plt.title("Top 10 Best Selling Products")
plt.xlabel("Total Quantity Sold")
plt.ylabel("Product Code")
plt.xticks(rotation = 45)
plt.grid()
plt.tight_layout()
plt.show()
```



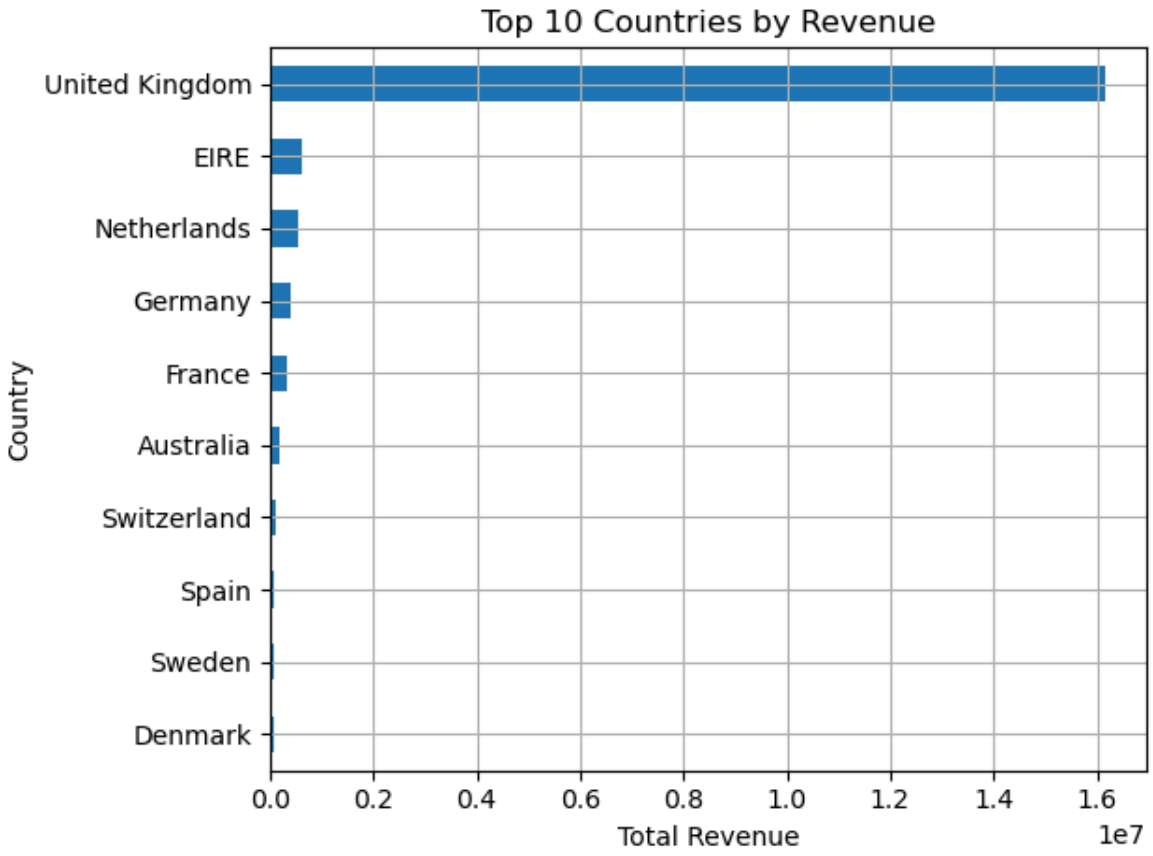
In [42]:
```
# Country-wise Revenue Analysis:-
country_revenue = df.groupby("Country")["Revenue"].sum()

# Get top 10 countries
top_countries = country_revenue.sort_values(ascending = False).head(10)

print(top_countries)
```

```
Country
United Kingdom    1.614474e+07
EIRE              6.099538e+05
Netherlands       5.483307e+05
Germany           4.119592e+05
France            3.217334e+05
Australia         1.664444e+05
Switzerland       9.940306e+04
Spain             9.101344e+04
Sweden            8.777552e+04
Denmark           6.445959e+04
Name: Revenue, dtype: float64
```

In [43]:
```
# (3) Visualization:-
top_countries.sort_values().plot(kind = "barh")
plt.title("Top 10 Countries by Revenue")
plt.xlabel("Total Revenue")
plt.ylabel("Country")
plt.xticks(rotation = 0)
plt.grid()
plt.tight_layout()
plt.show()
```



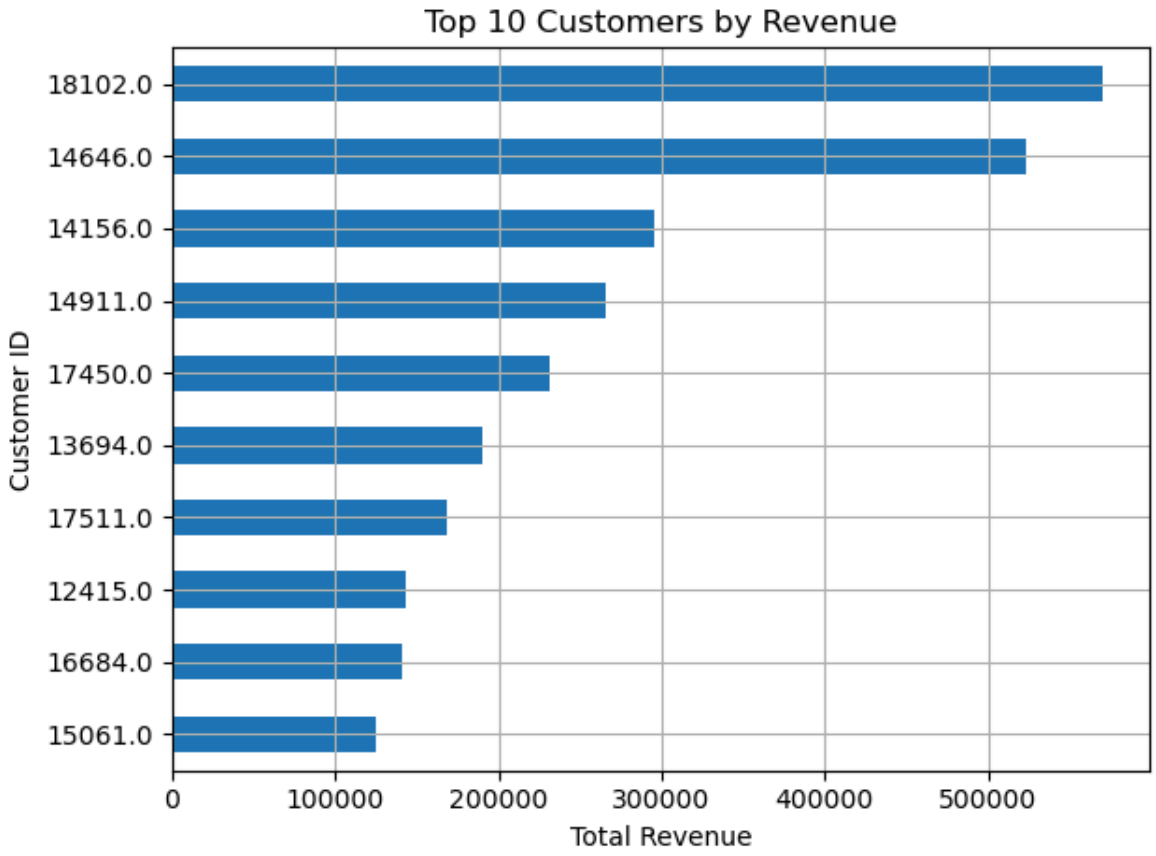
In [45]:
```python
# Customer Purchase Analysis:-
# Remove missing customer IDs for customer analysis:-
customer_data = df.dropna(subset=["Customer ID"])

# calculate revenue by customer:-
customer_revenue = customer_data.groupby("Customer ID")["Revenue"].sum()

# Top 10 customers by revenue:-
top_customers = customer_revenue.sort_values(ascending = False).head(10)
print(top_customers)
```

```
Customer ID
18102.0    570380.61
14646.0    523342.07
14156.0    296063.44
14911.0    265757.91
17450.0    231390.55
13694.0    190020.84
17511.0    168491.62
12415.0    143269.29
16684.0    141502.25
15061.0    124961.98
Name: Revenue, dtype: float64
```

In [46]:
```python
# (4) Visualization:-
top_customers.sort_values().plot(kind = "barh")
plt.title("Top 10 Customers by Revenue")
plt.xlabel("Total Revenue")
plt.ylabel("Customer ID")
plt.xticks(rotation = 0)
plt.grid()
plt.tight_layout()
plt.show()
```



In [47]:
```
# Order cancellation Analysis:-
# Identify cancelled invoices:-
cancelled_orders = df[df["Invoice"].astype(str).str.startswith("C")]

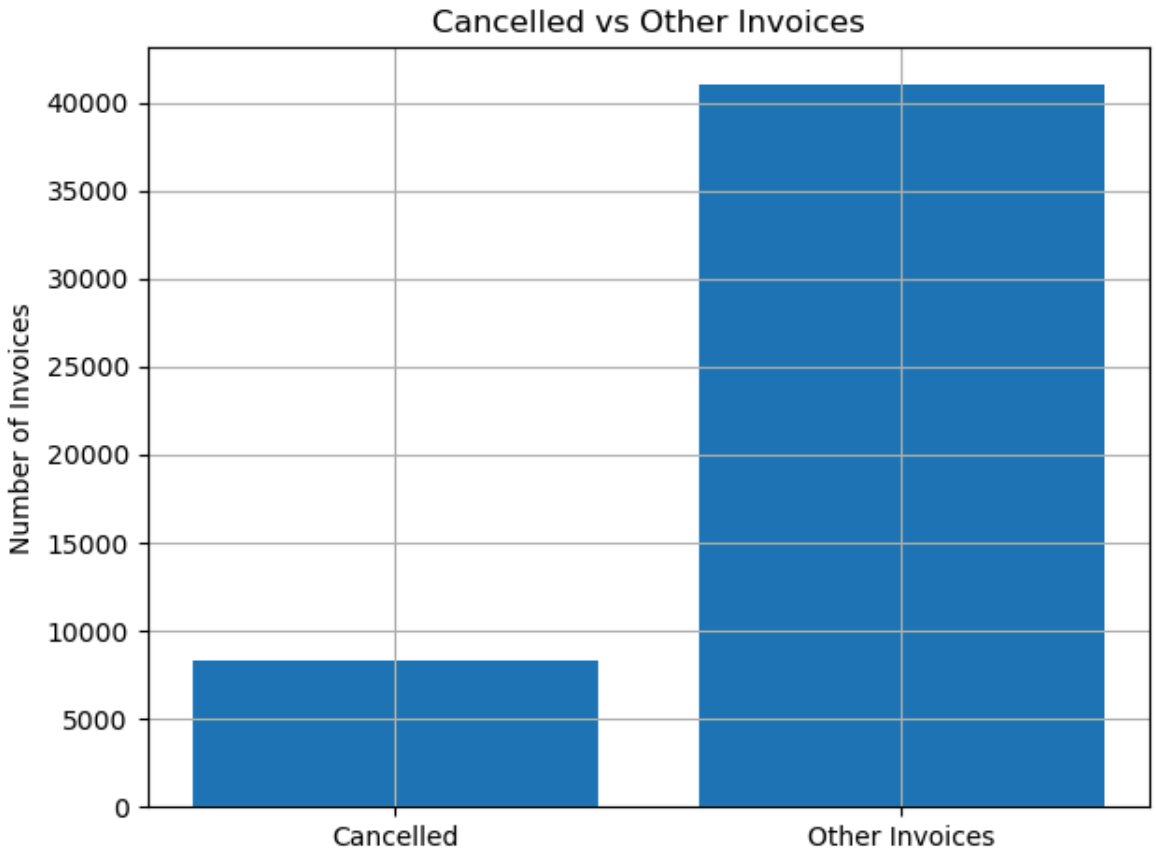
# Calculate cancellation KPIs:-
total_cancelled = cancelled_orders["Invoice"].nunique()
total_orders = df["Invoice"].nunique()

cancellation_rate = (total_cancelled / total_orders) * 100

print("Cancelled Invoices:-",total_cancelled)
print("Total Invoices:-",total_orders)
print("Cancellation Rate:-",round(cancellation_rate, 2),"%")
```

```
Cancelled Invoices:- 8292
Total Invoices:- 49348
Cancellation Rate:- 16.8 %
```

In [49]:
```
# (5) Visualization:-
labels = ["Cancelled","Other Invoices"]
values = [total_cancelled, total_orders - total_cancelled]
plt.bar(labels, values)
plt.title("Cancelled vs Other Invoices")
plt.ylabel("Number of Invoices")
plt.grid()
plt.tight_layout()
plt.show()
```



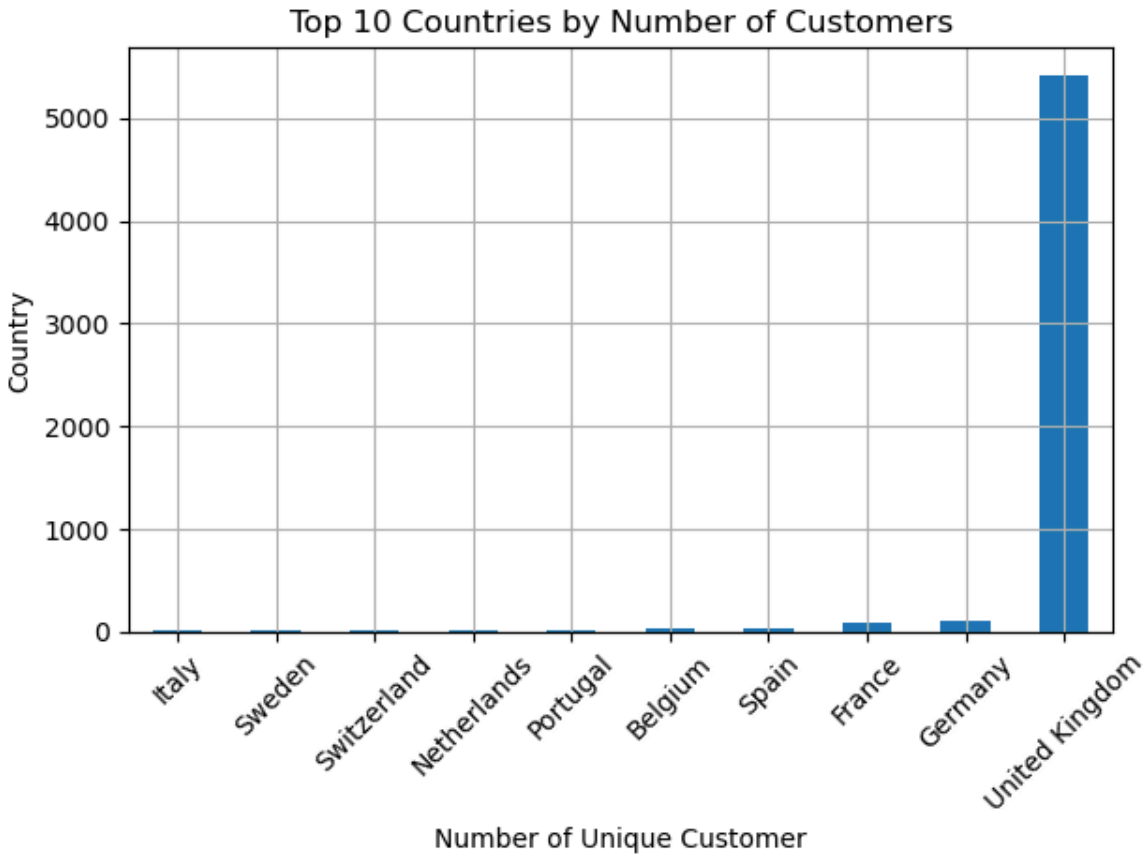
In [50]:
```
# Country-wise Customer Analysis:-
# Remove missing customer IDs:-
customer_data = df.dropna(subset = ["Customer ID"])

# Count unique customers by country:-
country_customers = customer_data.groupby("Country")["Customer ID"].nunique()

# Get top 10 countries:-
top_customer_countries = country_customer_countries = country_customers.sort_val
print(top_customer_countries)
```

```
Country
United Kingdom    5410
Germany            107
France              95
Spain               41
Belgium             29
Portugal            24
Netherlands         23
Switzerland         22
Sweden              19
Italy               17
Name: Customer ID, dtype: int64
```

In [53]:
```
# (6) Visualization:-
top_customer_countries.sort_values().plot(kind = "bar")
plt.title("Top 10 Countries by Number of Customers")
plt.xlabel("Number of Unique Customer")
plt.ylabel("Country")
plt.xticks(rotation = 45)
plt.grid()
plt.tight_layout()
plt.show()
```

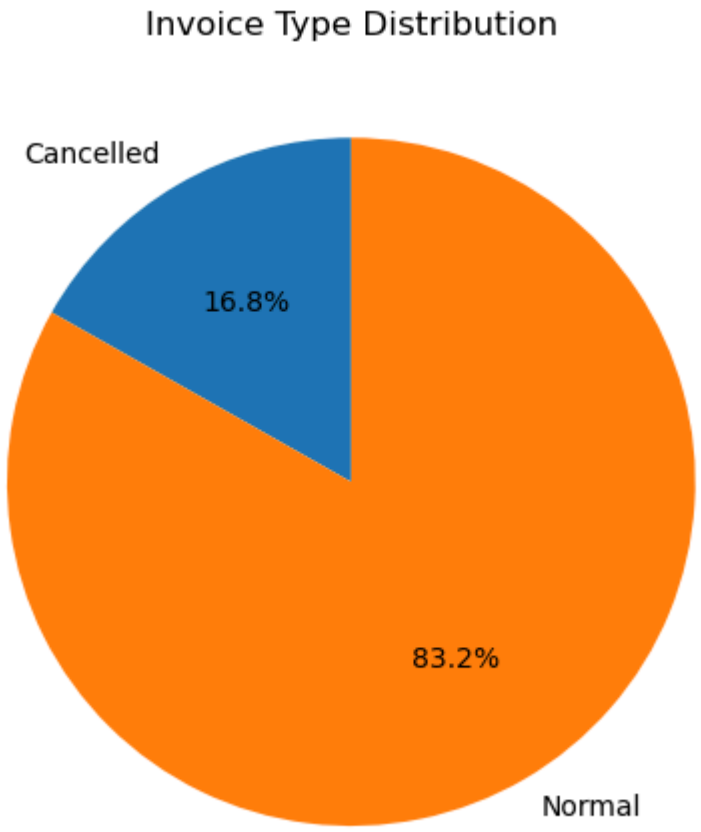


In [55]:
```
# Separate cancelled and non-cancelled invoices:-
invoice_type = df["Invoice"].astype(str).str.startswith("C")
cancelled = df.loc[invoice_type, "Invoice"].nunique()
normal = df.loc[~invoice_type, "Invoice"].nunique()

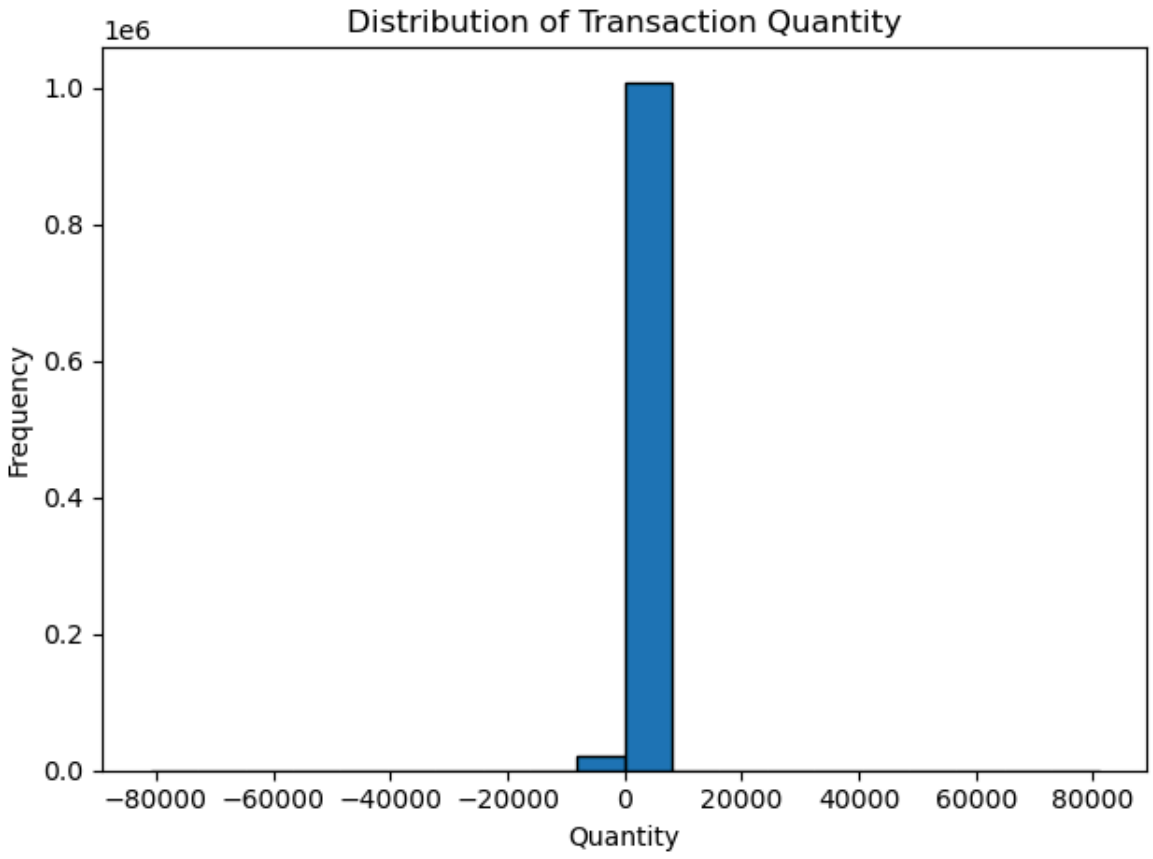
print("Cancelled Invoices:-",cancelled)
print("Normal Invoices:-",normal)
```

```
Cancelled Invoices:- 8292
Normal Invoices:- 41056
```

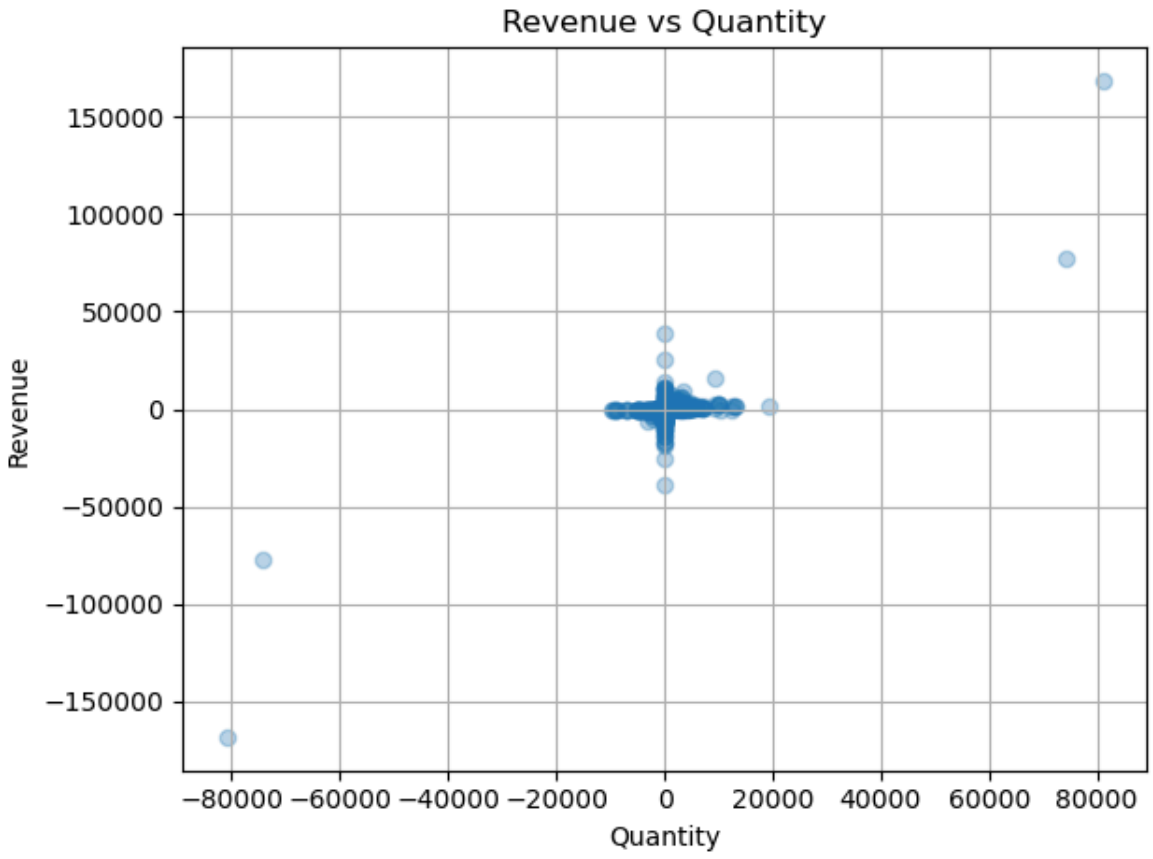
In [58]:
```
# (7) Visualization:-
plt.pie([cancelled, normal],labels=["Cancelled","Normal"],autopct = "%1.1f%%",st
plt.title("Invoice Type Distribution")
plt.tight_layout()
plt.show()
```



In [62]:
```
# Quantity Distribution:-
# (8) Visualization:-
plt.hist(df["Quantity"],bins=20,edgecolor="black")
plt.title("Distribution of Transaction Quantity")
plt.xlabel("Quantity")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



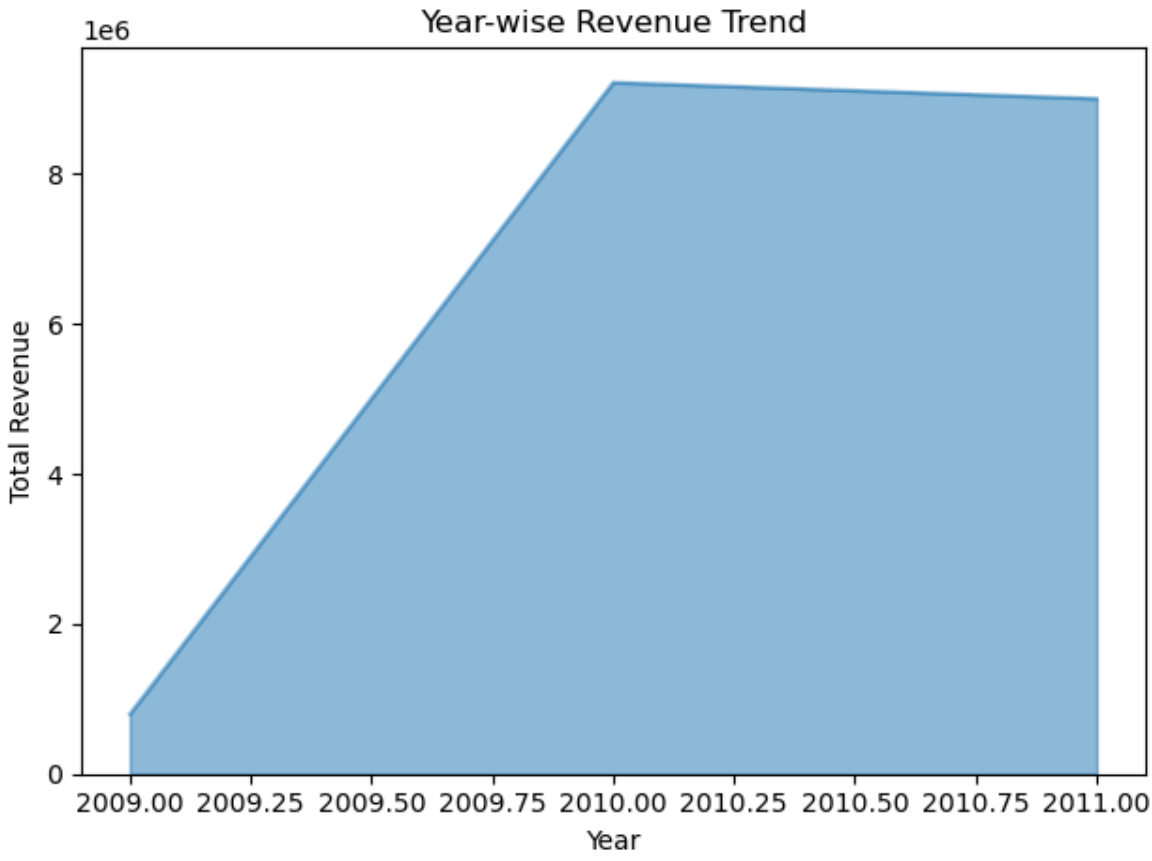
In [63]:
```
# Revenue vs Quantity Analysis:-
# (9) Visualization:-
plt.scatter(df["Quantity"],df["Revenue"],alpha = 0.3)
plt.title("Revenue vs Quantity")
plt.xlabel("Quantity")
plt.ylabel("Revenue")
plt.grid()
plt.tight_layout()
plt.show()
```



In [64]:
```
# Year-wise Revenue Analysis:-
# Calculate yearly revenue:-
yearly_revenue = df.groupby("Year")["Revenue"].sum()
print(yearly_revenue)
```

```
Year
2009     796648.500
2010    9216135.874
2011    9001425.464
Name: Revenue, dtype: float64
```

In [65]:
```
# (10) Visualization:-
yearly_revenue.plot(kind = "area",alpha=0.5)
plt.title("Year-wise Revenue Trend")
plt.xlabel("Year")
plt.ylabel("Total Revenue")
plt.tight_layout()
plt.show()
```



In [66]:
```
# Key Business Insights:-
```

In [67]:
```
# 1. Revenue Performance
#Revenue analysis helps identify the overall sales performance and changes in re

# 2. Product Performance
#Product analysis helps identify the products with the highest sales quantity.

# 3. Country-wise Performance
#Country-wise analysis identifies the countries contributing the most revenue an

# 4. Customer Analysis
#Customer analysis helps identify high-value customers and understand customer s

# 5. Order Cancellations and Returns
#Cancellation analysis helps understand cancelled invoices and the impact of ret

# 6. Monthly Sales Trends
#Monthly analysis helps identify changes in revenue across different months.

# 7. Quantity Distribution
#Quantity analysis helps identify common transaction quantities and unusually la
```

In [68]:
```
# Business Recommendations:-

#1. Focus on products that demonstrate strong sales performance.

#2. Develop targeted marketing strategies for important markets.

#3. Monitor cancelled invoices and product returns to identify potential operati

#4. Study customer purchasing patterns to improve customer retention.
```

```
#5. Monitor monthly revenue trends to support better business planning.
```

In [69]:
```
# Project Conclusion:-

#The Online Retail II project analyzes retail transaction data using Python, Pan

#The analysis covers revenue performance, product sales, customer behavior, coun

#The insights can help businesses understand sales patterns,identify important p
```

In [70]:
```
# Project Workflow:-

#✅ Data Cleaning
#✅ Exploratory Data Analysis (EDA)
#✅ Overall Business KPIs
#✅ Revenue and Product Analysis
#✅ Customer and Country Analysis
#✅ Cancellation Analysis
#✅ Multiple Visualizations
#✅ Business Insights – initial draft
#✅ Business Recommendations
#✅ Project Conclusion
```

In [ ]:
```

```